DESIGN A TIMER-BASED LED CONTROL SYSTEM USING ESP32

A MINI PROJECT REPORT

Submitted by

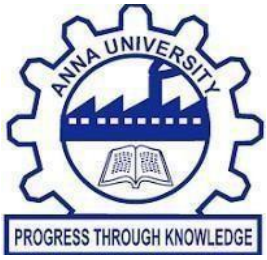
A. KALAIYARASI 312325106068

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING





St. JOSEPH'S COLLEGE OF ENGINEERING (An AUTONOMOUS INSTITUTION) CHENNAI - 600 119

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026



You Choose, We Do It

St. JOSEPH'S COLLEGE OF ENGINEERING

(An Autonomous Institution)

St. Joseph's Group of Institutions

OMR, CHENNAI - 119



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BATCH: 2025-2029 (III SEM)

MINI PROJECT EVALUATION SHEET

Project Title	**:**	
Student Name	**:**	
Register Number	**:**	
Date of Evaluation	**:**	

Evaluation Criteria

S.No	Criteria	Maximum Marks	Marks Awarded
1	**Project Concept & Relevance**	10	
2	**Design & Implementation**	10	
3	**Presentation & Documentation**	10	
	Total	30	
	Total		

Evaluator Name and Signature

ABSTRACT

This project presents the design of a timer-based LED control system using the ESP32 microcontroller. The system controls eight LEDs through eight individual current limiting resistors. The ESP32 is programmed to switch the LEDs ON and OFF according to predefined time intervals. This project demonstrates the use of embedded programming, digital output pins and timing functions to automate lighting sequences. Eight LEDs are connected to selected ESP32 GPIO pins, with one resistor connected in series with each LED to limit current. The program can switch the LEDs together or operate them in a sequence, depending on the selected timing pattern. Delay-based timing or non-blocking timer logic can be used to set the ON and OFF duration. The LEDs provide a simple
visual indication of the controller's output states.

The software is developed using the Arduino IDE and the ESP32 board package. The program configures the eight GPIO pins as outputs and repeatedly executes the timing sequence. Each LED state is changed after the specified time interval. The circuit can first be checked in a suitable simulator, then assembled on a breadboard and tested by observing the LED sequence and timing.

The proposed system is low-cost, easy to understand and suitable for learning microcontroller interfacing and automation. The timing values and LED patterns can be modified in software. Future improvements may include push-button control, adjustable timing, an LCD or OLED display, Wi-Fi control and use of a real-time clock for scheduled operation.

Keywords: ESP32, Timer-Based Control, LED, GPIO, Resistor, Embedded Systems, Automation.

TABLE OF CONTENTS

Chapter No.	Title	Page
	Acknowledgement	iv
	Abstract	v
	List of Figures / List of Tables	ix / x
	SDG Goals Mapping	xii
1	INTRODUCTION	1
	1.1 Background and Motivation	1
	1.2 Technological Approach	2
	1.3 Challenges and Vision	2
	1.4 Objectives	3
	1.5 Organisation of the Report	3
	1.6 Conclusion	4
2	LITERATURE SURVEY	5
	2.1 Evolution of Embedded Access Control Systems	5
	2.2 Keypad-Based Password Systems	5
	2.3 Advanced Authentication Techniques	5
	2.4 Conclusion	5
3	METHODOLOGY	6
	3.1 Limitations of Existing Systems	6
	3.2 Advantages of the Proposed System	6
	3.3 Flowchart	7
	3.4 System Architecture (Block Diagram)	8
	3.5 Hardware Description	8
	3.6 Software Description	9
	3.7 Circuit Connections and Working Principle	10
	3.8 Performance Evaluation Parameters	10
	3.9 Conclusion	11

4	RESULTS AND DISCUSSION	12
	4.1 Experimental Setup	12
	4.2 Simulation Results	12
	4.3 Hardware Implementation Results	12
	4.4 Test Cases and Observations	12
	4.5 Response Time and Reliability Analysis	13
	4.6 Conclusion	13
5	CONCLUSION	14
	5.1 Conclusion	14
	5.2 Suggestions for Future Work	14
	References	15
	Appendix A (ESP32 Program, Component List)	15

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO
3.1	Flowchart of the Timer-Based LED Control System Using ESP32	10
3.2	Block Diagram of the System Architecture	11
3.3	Circuit Diagram of the ESP32 with Keypad, LCD, LED and Buzzer	14
4.1	Simulation Output: LED OFF (Initial State, "Enter Password")	17
4.2	Simulation Output: Wrong Password Entered (Buzzer ON, LED OFF)	17
4.3	Simulation Output: Correct Password Entered (LED ON, "Access Granted")	18
4.4	Hardware Prototype of the Timer-Based LED Control System Using ESP32	18
4.5	LCD Display Messages During Different Operating States	19
4.6	System Lockout After Three Wrong Attempts	2

LIST OF TABLE

Table No.	Title	Page No.
3.1	Components Used in the Proposed System	12
3.2	Pin Connections between ESP32 and Peripherals	14
4.1	Test Cases and Observations	19
4.2	Response Time Measurements	20

LIST OF ABBREVIATIONS

- LED (Light Emitting Diode)
- LCD (Liquid Crystal Display),
- I2C (Inter-Integrated Circuit),
- EEPROM (Electrically Erasable Programmable Read-Only emory)

- ☒ IDE (Integrated Development Environment)
- ☒ GPIO (General Purpose Input/Output)
- ☒ PWM (Pulse Width Modulation)RFID (Radio Frequency Identification)

x

MAPPING OF SDG

Project Title: Design a Timer-Based LED Control System Using ESP32

	SDG 1	SDG 2	SDG 3	SDG 4	SDG 5	SDG 6	SDG 7	SDG 8	SDG 9	SDG 10	SDG 11	SDG 12	SDG 13	SDG 14	SDG 15	SDG 16	SDG 17
High (3)																	
Medium (2)																	
Low (1)																	

SDG 1 – No Poverty | **SDG 2** – Zero Hunger | **SDG 3** – Good Health and Well-Being | **SDG 4** – Quality Education | **SDG 5** – Gender Equality | **SDG 6** – Clean Water and Sanitation | **SDG 7** – Affordable and Clean Energy | **SDG 8** – Decent Work and Economic Growth | **SDG 9** – Industry, Innovation and Infrastructure | **SDG 10** – Reduced Inequalities | **SDG 11** – Sustainable Cities and Communities | **SDG 12** – Responsible Consumption and Production | **SDG 13** – Climate Action | **SDG 14** – Life Below Water | **SDG 15** – Life on Land | **SDG 16** – Peace, Justice and Strong Institutions | **SDG 17** – Partnerships for the Goals

SDG	Level	Justification
SDG 9 – Industry, Innovation and Infrastructure	High (3)	Embedded electronics and automation for secure, modern infrastructure
SDG 11 – Sustainable Cities and Communities	Medium (2)	Basis for smart home and building access systems
SDG 4 – Quality Education	Medium (2)	Hands-on learning of embedded systems
SDG 7 – Affordable and Clean Energy	Low (1)	Low-power design and controlled usage of devices

CHAPTER 1 INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Lighting and indicator systems are widely used in homes, industries, laboratories and electronic equipment. Manual switching is suitable for simple applications, but it cannot provide automatic switching at fixed time intervals without additional control. A timerbased controller can switch lights according to a programmed schedule and can repeat a sequence continuously.

Microcontrollers make timer-based automation practical because they combine processing, memory and digital input/output in a small development board. The ESP32 provides multiple GPIO pins and a programmable timing environment, making it suitable for controlling several LEDs. In this project, eight LEDs represent eight output channels and demonstrate how a programmed controller can manage multiple indicators.

The project uses an ESP32 development board, eight LEDs, eight current-limiting resistors, a breadboard and jumper wires. The ESP32 executes a program that changes the output states after defined intervals. Each LED is connected through its own resistor to protect the LED and limit current. The same principle can later be extended to low-voltage indicator panels or, with appropriate driver hardware, larger loads.

1.2 TECHNOLOGICAL APPROACH

The proposed system follows an input–process–output approach. The ESP32 acts as the processing unit, while eight GPIO pins provide the output signals. The program defines the LED pins, configures them as outputs and sets the timing intervals for the selected sequence.

At the start, the controller places the LEDs in their initial states. After each specified interval, it changes one or more GPIO outputs to HIGH or LOW. The LEDs respond to these output states, producing a timed ON/OFF pattern. The timing can be adjusted in the program to create different sequences or durations.

The software is written in C/C++ in the Arduino IDE using the ESP32 board support package. For a simple demonstration, the delay() function can provide fixed intervals. For more responsive designs, millis() can be used to track elapsed time without stopping the main program. The program is uploaded to the ESP32 through USB and tested by observing all eight outputs.

1.3 CHALLENGES AND VISION

The main design challenges are selecting suitable GPIO pins, wiring each LED with a series resistor, ensuring a common ground and choosing timing intervals that are easy to observe. The ESP32 uses 3.3 V GPIO logic, so its pins must not be connected to loads that exceed their electrical limits. Each LED needs a resistor, and external driver circuitry is required if the project is extended to higher-current loads.

Another consideration is timing accuracy. A simple delay-based program is easy to understand but pauses other program tasks while waiting. A millis()-based approach allows the controller to check elapsed time and update LEDs without blocking. Correct wiring, suitable resistor values and stable power are important for reliable operation.

The project can be extended with adjustable timing, push buttons, an LCD/OLED display, an RTC module for clock-based schedules, or Wi-Fi control using the ESP32's wireless capability. A driver stage can also be added to operate suitable external loads safely. These extensions are optional and should only be described as implemented if they are actually built.

1.4 OBJECTIVES

The primary objectives of this project are:

1. **To design and implement a timer-based LED control system using the ESP32 microcontroller.**
2. **To interface eight LEDs and eight current-limiting resistors with ESP32 GPIO pins.**
3. **To write a program that switches LED outputs according to predefined time intervals.**
4. **To observe and verify the timing sequence and output states of all eight LEDs.**
5. **To understand GPIO configuration, timing logic and embedded C/C++ programming.**
6. **To test the circuit and identify wiring or timing errors.**
7. **To identify possible extensions for scheduled lighting and automation applications.**

1.5 ORGANISATION OF THE REPORT

This report presents the design, implementation and evaluation of a timer-based LED control system using ESP32 and eight LED output channels.

Chapter 1 introduces timer-based control, the project motivation, the proposed approach, challenges and objectives.

Chapter 2 reviews microcontroller-based timing and LED control concepts, including GPIO operation and timer programming.

Chapter 3 explains the proposed methodology, system architecture, components, software, circuit connections and working sequence.

Chapter 4 describes the test setup, expected observations, test cases and timing evaluation. Actual measurements should be entered after testing.

Chapter 5 summarises the project and lists possible future improvements such as adjustable timing, display integration and wireless control.
The report ends with references and an appendix containing example ESP32 program logic and component details.

1.6 CONCLUSION

Microcontroller-based control systems are widely used for timed switching, indicator sequencing and automation. A microcontroller can configure digital pins as outputs and change their logic levels through software. The 8051 family, AVR boards and modern ESP32 boards can all demonstrate these principles, although their pin capabilities, voltage levels and programming environments differ.

The ESP32 is useful for student projects because it provides GPIO, programmable timing and integrated wireless capability. A simple eight-LED circuit provides a safe, visible way to study digital output control. Reliable operation depends on correct pin mapping, individual current-limiting resistors and a suitable timing algorithm.

CHAPTER 2 LITERATURE SURVEY

2.1 EVOLUTION OF EMBEDDED ACCESS CONTROL SYSTEMS

Earlier electronic control circuits used discrete timers and logic circuits to switch indicators at fixed intervals. Microcontrollers simplified these designs by allowing timing sequences to be changed through software rather than hardware modifications.

Modern boards such as the ESP32 provide digital input/output pins and software timing functions. A program can set an output HIGH or LOW and use elapsed-time checks to determine when the next state change should occur. This approach is commonly applied to LED sequencing, warning indicators and basic automation demonstrations.

2.2 KEYPAD-BASED PASSWORD SYSTEMS

LED control projects are commonly used to introduce GPIO programming. An LED connected to a digital output changes state when the controller writes HIGH or LOW, provided the LED is wired with correct polarity and a suitable series resistor.

GPIO and timing. Each LED uses one output pin, so eight LEDs require eight available GPIO pins. The software defines the pins in an array and iterates through them to apply a sequence.
Timing may be implemented with delay() for a simple demonstration or millis() for a nonblocking sequence.

Resistor selection. A series resistor limits LED current. The resistor value depends on supply
voltage, LED forward voltage and desired current.

2.3 CONCLUSION

Timer-based LED control is a straightforward introduction to embedded systems. Eight output channels make the sequence easy to observe, while adjustable intervals demonstrate how software determines the behaviour of hardware. More advanced versions can add user inputs, display modules or real-time scheduling.

CHAPTER 3
METHODOLOGY

3.1 LIMITATIONS OF EXISTING SYSTEMS

Manual switches do not automatically change state at predefined intervals. A purely hardware timer circuit may require component changes to alter its sequence. The proposed microcontroller design allows the timing and output order to be modified in software.

Common implementation problems include reversed LED polarity, missing resistors, incorrect GPIO assignments, poor breadboard connections and timing values that are too short to observe. GPIO pins also have electrical limits, so LEDs must be wired correctly and higher-current loads must use suitable driver circuitry.

3.2 ADVANTAGES OF THE PROPOSED SYSTEM

- It uses an ESP32 board and commonly available components.
- Eight LEDs provide eight visible output channels.
- Each LED has an individual current-limiting resistor.
- The ON/OFF sequence and interval can be modified in software.
- The program demonstrates GPIO configuration and timing logic.
- The design can be extended with buttons, a display or a driver circuit.
- The ESP32 also provides Wi-Fi and Bluetooth capability for optional future extensions.

3.3 FLOWCHART

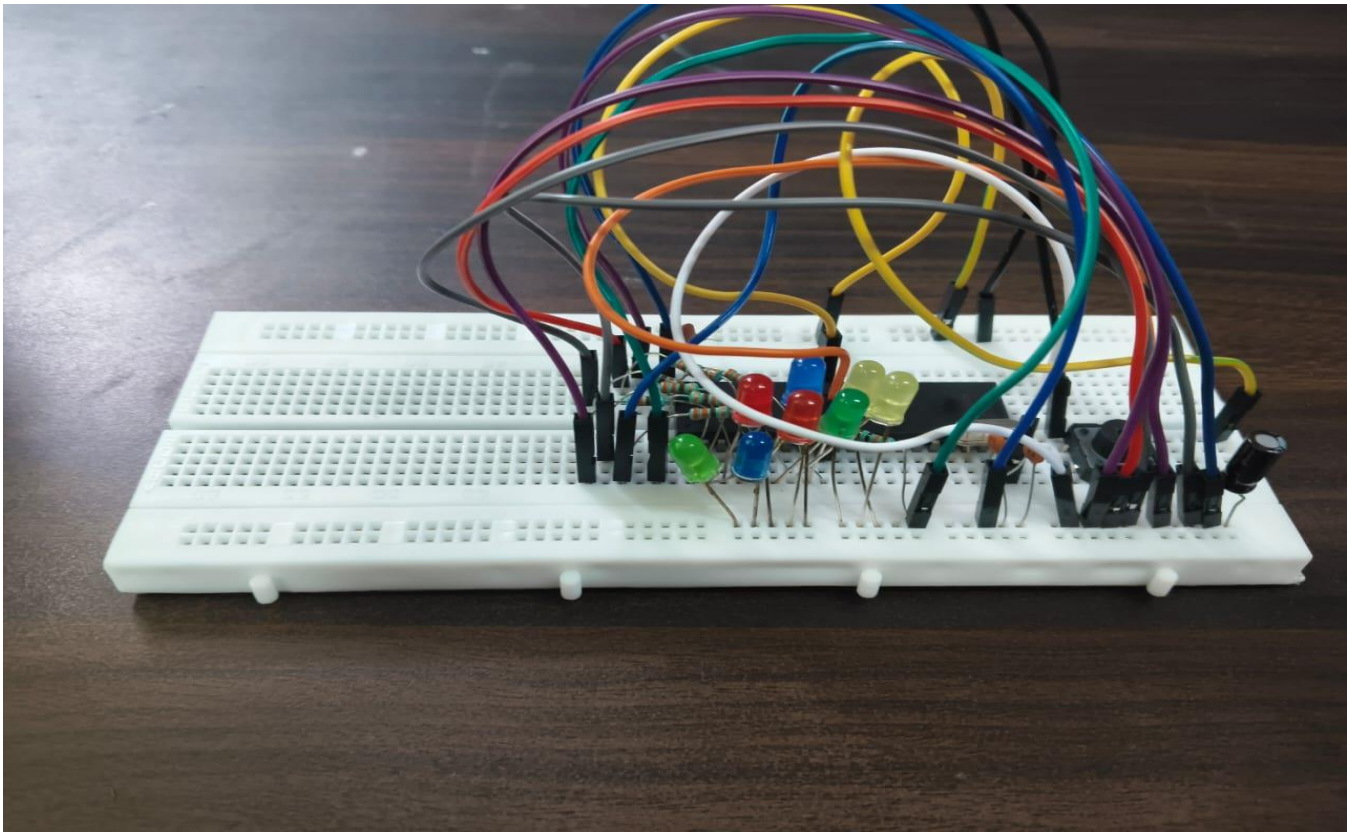


Fig 3.1 Workflow Diagram

WORKFLOW:

1. Power on the ESP32 and initialise the eight LED GPIO pins as outputs.
2. Set all LEDs to the initial OFF state.
3. Load the programmed time interval and sequence index.
4. Wait until the selected time interval has elapsed.
5. Change the required LED output state.
6. Move to the next LED or next programmed sequence step.
7. Repeat the sequence until the controller is powered off or the program is stopped.
8. Observe the ON/OFF state of each LED and confirm the order.
9. If timing is incorrect, check the interval value and program logic

3.4 SYSTEM ARCHITECTURE

The system consists of a processing block and an output block. The ESP32 runs the program and controls eight LED channels.

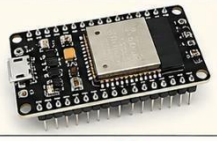
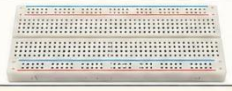
The processing block is the ESP32 development board. It configures GPIO pins and determines when each output should change state.

The output block contains eight LEDs, each connected in series with a current-limiting resistor. The LEDs indicate the current output pattern.

A breadboard and jumper wires provide the circuit connections. The ESP32 and external circuit must share ground, and each LED must have its own resistor.

The board is powered through its USB connector or a suitable regulated supply. Use only the board's recommended power input.

3.5 HARDWARE DESCRIPTION

Component	Specification	Purpose
ESP32 Development Board 	ESP32-WROOM-32 (ESP32 DevKit V1), 3.3 V operation, 30 GPIO pins, Wi-Fi & Bluetooth	Main controller for timer-based LED control
LED (8 Nos.) 	Red colour, 5 mm, 8 LEDs	Output indicators (8 LEDs blink together)
Resistor (8 Nos.) 	220 Ω, 1/4 W, 8 resistors	Limits current through each LED
Breadboard 	Standard size	Circuit assembly and prototyping
Jumper Wires 	Male-to-male	Electrical connections between components
USB Cable 	Compatible with ESP32 board	Power supply and programming
Computer / Laptop 	Arduino IDE installed (Windows / Linux / macOS)	Writing, compiling and uploading the program

ESP32 development board. The ESP32 is a microcontroller-based development platform with digital GPIO pins, program memory, timers and integrated wireless capability. GPIO logic is 3.3 V, so wiring must follow the board specifications.

LEDs. Eight LEDs are used as visual output indicators. Observe polarity: the anode is connected toward the GPIO output through a resistor, and the cathode is connected to ground, for the active-HIGH arrangement.

Resistors. Eight resistors are required, one in series with each LED. A typical starting value such as 220 Ω to 330 Ω may be used depending on the LED and current target; verify the current and ESP32 board limits before powering the circuit. The breadboard allows the ESP32, resistors and LEDs to be connected without soldering.

Jumper wires. Jumper wires connect the GPIO pins, resistors, LEDs and ground rails.

3.6 SOFTWARE DESCRIPTION

The program is written in C/C++ using the Arduino IDE and the ESP32 board package.

The main tasks are:

- Define an array containing the eight selected GPIO pin numbers.
- Configure every LED pin as OUTPUT and set the initial state to LOW.
- Use a timer interval to determine when the next LED state change should occur. The setup() function runs once and configures the pins. The loop() function repeatedly checks elapsed time and updates the outputs. For a simple version, delay() can be used; for a non-blocking version, millis() tracks elapsed time.

3.7 CIRCUIT CONNECTIONS AND WORKING PRINCIPLE

Component	ESP32 GPIO Pin / Connection
LED 1 (through 220 Ω resistor)	GPIO 23
LED 2 (through 220 Ω resistor)	GPIO 22
LED 3 (through 220 Ω resistor)	GPIO 21
LED 4 (through 220 Ω resistor)	GPIO 19
LED 5 (through 220 Ω resistor)	GPIO 18
LED 6 (through 220 Ω resistor)	GPIO 17
LED 7 (through 220 Ω resistor)	GPIO 16
LED 8 (through 220 Ω resistor)	GPIO 4
LED cathodes (all eight)	Common GND
ESP32 power/programming	USB cable / USB port

When power is applied, the ESP32 configures all eight LED pins as outputs and turns the LEDs off. The program then waits for the chosen interval and starts the programmed sequence.

At each timing event, the controller changes the selected GPIO output. The corresponding LED turns ON or OFF according to the program. The sequence repeats, making the timer-controlled operation visible.

The interval and sequence can be modified in the program. If an LED does not respond, check GPIO selection, polarity, resistor placement, ground connection and the uploaded code.

3.8 PERFORMANCE EVALUATION PARAMETERS

The system is evaluated on the following parameters:

- Functional correctness: whether all eight LEDs follow the intended sequence.
- Timing behaviour: whether each state changes after the programmed interval.
- Output accuracy: whether the correct GPIO pin controls each LED.
- Reliability: whether the sequence repeats consistently.
- Electrical safety: whether each LED has a resistor and the GPIO limits are respected.
- Power stability: whether the board and LEDs operate without unexpected resets.

3.9 CONCLUSION

This chapter described the system limitations, advantages, workflow, architecture, hardware, software and working principle. The next chapter outlines the testing approach and observations.

CHAPTER 4 RESULTS AND DISCUSSIONS

4.1 EXPERIMENTAL SETUP

The proposed circuit uses an ESP32 development board, eight LEDs, eight current-limiting resistors, a breadboard and jumper wires. The program is uploaded using the Arduino IDE. Record the actual board model, resistor values and timing interval used in the final hardware.

4.2 SIMULATION RESULTS

Before hardware testing, verify the pin assignments and program logic. At start-up, all LEDs should be in the defined initial state. After each programmed interval, the corresponding LED output should change according to the sequence.

4.3 HARDWARE IMPLEMENTATION RESULT

After assembling the circuit, power the ESP32 and observe the eight LEDs. Confirm that each LED responds to its assigned GPIO and that the sequence repeats. Enter the actual observed results and photographs from the working prototype in the final report.

4.4 TEST CASES AND OBSERVATIONS

Table 4.1: Test Cases for Timer-Based LED Control System Using ESP32

Test No.	Input / Condition	Expected Output	Observed Output	Result
1	Power on ESP32	All 8 LEDs start in the programmed initial state (OFF)	[Fill after testing]	[Pass/Fail]
2	Timer interval reached	All 8 LEDs turn ON together	[Fill after testing]	[Pass/Fail]
3	Next timer interval reached	All 8 LEDs turn OFF together	[Fill after testing]	[Pass/Fail]
4	Repeated timer cycles	All 8 LEDs continue blinking together in the same sequence	[Fill after testing]	[Pass/Fail]
5	Change programmed interval	All 8 LEDs blink at the updated interval after re-uploading code	[Fill after testing]	[Pass/Fail]
6	Check LED wiring and GPIO assignments	Each of the eight LEDs responds correctly and no LED remains incorrectly lit	[Fill after testing]	[Pass/Fail]

4.5 RESPONSE TIME AND RELIABILITY ANALYSIS

Trial	Time from timer event to LED response (ms)
1	[ ]
2	[ ]
3	[ ]
4	[ ]
5	[ ]
Average	[ ]

The timing interval should be measured using the actual program and hardware if accurate performance values are required. Test repeated cycles and record whether the LEDs change in the correct order. Do not report unmeasured response times or trial counts as experimental results.

4.6 CONCLUSION

The system is expected to demonstrate timer-based control of eight LED outputs. The final result section should be completed using observations from the actual circuit, including the selected interval, sequence order and any wiring or timing issues found during testing.

CHAPTER 5 CONCLUSION

5.1 CONCLUSION

This project presents a timer-based LED control system using the ESP32 microcontroller. Eight LEDs are controlled through eight individual current-limiting resistors, and their output states change according to a programmed timing sequence.

The design demonstrates GPIO interfacing, embedded C/C++ programming and timer logic. It is inexpensive and easy to modify, and can be extended with adjustable timing, displays, push buttons or wireless control. Final performance claims should be based on measurements from the completed prototype.

5.2 SUGGESTIONS FOR FUTURE WORK

Several improvements can be made to this system:

- Add push buttons to select the timing interval or operating mode.
- Add an LCD or OLED display to show the active sequence and interval.
- Use millis()-based timing to avoid blocking other program tasks.
- Add an RTC module for operation based on real clock time.
- Use the ESP32 Wi-Fi or Bluetooth capability for optional wireless control.
- Add suitable transistor or driver circuitry if controlling higher-current loads.

REFERENCES

1) **Espressif Systems, "ESP32 Series Datasheet and Technical Documentation." [Online]. Available: https://www.espressif.com/support/documents/technical documents**

2) **Arduino, "Arduino IDE Documentation." [Online]. Available: https://docs.arduino.cc/**

3) **Espressif Systems, "Arduino-ESP32 Documentation." [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/**

4) **Espressif Systems, "ESP32 Hardware Design Guidelines." [Online]. Available: https://docs.espressif.com/projects/esp-hardwaredesignguidelines/en/latest/**

5) **S. Monk, *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education.**

6) **M. Margolis, *Arduino Cookbook*. O'Reilly Media.**

7) **Purdue Online Writing Lab, "IEEE Reference List." [Online]. Available: https://owl.purdue.edu/owl/research_and_citation/ieee_style/reference_list.html**

Appendix A: Sample Arduino Code

```
/*
  Project Title:
  Design of a Timer-Based LED Control System Using ESP32

  Hardware:
1. ESP32 DevKit V1
2. Eight LEDs
3. Eight 220-ohm resistors
4. Breadboard and jumper wires

  Operation:
  All eight LEDs turn ON together for 1 second
and turn OFF together for 1 second.
```

```
  The cycle repeats continuously.
*/

const int ledPins[8] = { 13,
14, 16, 17, 18, 19, 21, 22
};

// LED switching interval: 1000 milliseconds const
unsigned long intervalMs = 1000;

// Store the previous timer value unsigned
long previousMillis = 0;

// Initial LED state bool
ledState = false;

void setup() {
  // Configure all eight LED pins as outputs
for (int i = 0; i < 8; i++) {
pinMode(ledPins[i], OUTPUT);

   // Initially switch OFF all LEDs
digitalWrite(ledPins[i], LOW);
  }
}

void loop() {
  // Read the current elapsed time
unsigned long currentMillis = millis();    //
Check whether the timer interval has
elapsed if (currentMillis - previousMillis
>= intervalMs) {


   // Update the timer reference
previousMillis = currentMillis;     // Toggle
the
LED state     ledState = !ledState;
```

```
   // Apply the same state to all eight
LEDs     for (int i = 0; i < 8; i++) {
digitalWrite(        ledPins[i],         ledState
? HIGH : LOW
     );
   }
  }
}
```